

MF0487_3

EXAMEN

AUDITORÍA

DE SEGURIDAD

INFORMÁTICA

Alumna: Paloma Casabona

Realizar la auditoría de seguridad de la IP proporcionada (3.88.28.22) y completar el documento con todas las evidencias encontradas. Una vez que termineis, subirlo a GH y enviar el enlace del doc.

MegaCorp web server

Página por defecto. Este servidor aloja varios **hosts virtuales por nombre** (*name-based virtual hosts*).

Has llegado a esta página **por IP**, así que estás viendo el sitio por defecto (*fallback*). Las aplicaciones reales viven en nombres de host distintos y se seleccionan mediante la cabecera HTTP `Host` — no existe DNS público para ellas.

Objetivo del examen

Realiza una **auditoría de seguridad autorizada** de este *único host*. Hay **flags** con el formato `FLAG{...}` repartidas por los servicios; cada una corresponde a una vulnerabilidad o mala configuración real. Documenta cada hallazgo en el **informe**: qué es, cómo lo encontraste (herramienta + comando), la evidencia (flag) y una mitigación.

Cómo llegar a los sitios reales

Este servidor sirve *varios* sitios detrás de esta misma IP. Elige cuál ver poniendo la cabecera `Host:` — no hace falta DNS, la fijas tú.

Cliente / Objetivo: MegaCorp — 3.88.28.22 / megacorp.lab

Tipo de auditoría: Caja negra, externa

Auditor(es): Paloma Casabona Agudo

Fecha de inicio: 2026/07/02

Fecha de fin: 2026/07/02

Versión del informe: 1.0

Clasificación: Confidencial

1. RESUMEN EJECUTIVO

Contexto y objetivo de la auditoría

La presente auditoría de seguridad tuvo como objetivo evaluar la exposición de los servicios publicados por **MegaCorp**, identificando vulnerabilidades de configuración, errores de despliegue y posibles riesgos derivados de la divulgación de información sensible. Para ello se realizaron tareas de reconocimiento, enumeración y verificación manual de los servicios accesibles desde la red.

Valoración global del riesgo

ALTO — Se identificaron múltiples vulnerabilidades de configuración que permitieron acceder a recursos internos, descubrir información sensible sobre la infraestructura y localizar repositorios de código fuente expuestos, incrementando significativamente la superficie de ataque.

Principales conclusiones

- Se identificaron varios **hosts virtuales internos** accesibles desde Internet mediante la modificación de la cabecera **Host**.
- El certificado TLS revelaba información interna de la infraestructura, permitiendo descubrir nuevos servicios y dominios.
- Se localizaron **repositorios Git expuestos**, con riesgo de divulgación del código fuente y de información sensible.
- Se detectó un **panel de monitorización interno accesible sin autenticación** y servicios internos publicados en puertos no estándar.
- Se observaron múltiples deficiencias de configuración y falta de medidas de **hardening** en los servicios auditados.

Recomendaciones prioritarias

1. Eliminar los repositorios **.git** y cualquier información sensible expuesta en los servidores de producción.
2. Restringir el acceso a servicios internos mediante autenticación, segmentación de red y reglas de firewall.
3. Revisar la configuración de los servidores web y certificados TLS, aplicando medidas de **hardening** y deshabilitando el listado de directorios.

Resumen de hallazgos por severidad

Severidad	N.º de hallazgos
Crítica	0
Alta	4
Media	6
Baja	1
Informativa	0
TOTAL	11

Esta distribución (0 críticas, 4 altas, 6 medias y 1 baja) es consistente con los hallazgos que habéis documentado: la mayoría son errores de configuración con impacto importante, pero sin evidencia de ejecución remota de código o compromiso total del sistema, por lo que una valoración global de **riesgo Alto** está bien justificada.

2. ALCANCE Y REGLAS DE ENFRENTAMIENTO

- **Activos dentro de alcance:** Dirección IP **3.88.28.22** y todos los servicios asociados (HTTP, HTTPS, FTP, Redis y demás servicios expuestos), incluyendo los hosts virtuales descubiertos (www.megacorp.lab, dev.megacorp.lab, admin.megacorp.lab, api.megacorp.lab, internal.megacorp.lab, staging.megacorp.lab y vpn.megacorp.lab).
- **Activos fuera de alcance:** Cualquier sistema, red o servicio no asociado a la dirección IP objetivo o no autorizado expresamente para la auditoría.

- **Ventana temporal autorizada:** Durante el desarrollo de la práctica/examen, dentro del tiempo establecido para su realización.
- **IP(s) de origen del auditor:** Equipo Kali Linux del auditor (dirección IP asignada dentro del entorno de prácticas).
- **Restricciones (RoE):**
 - No realizar ataques de denegación de servicio (DoS/DDoS).
 - No efectuar ataques de fuerza bruta contra servicios de autenticación.
 - No modificar, eliminar ni alterar información del sistema objetivo.
 - No pivotar hacia otros sistemas o redes fuera del alcance definido.
 - Limitar las pruebas a técnicas de reconocimiento, enumeración y verificación de vulnerabilidades autorizadas.

Limitaciones encontradas

Durante la auditoría no fue posible evaluar servicios que no estaban expuestos públicamente ni aquellos que requerían credenciales válidas. Las pruebas se limitaron a la información y servicios accesibles desde el entorno autorizado de prácticas, respetando en todo momento las reglas de enfrentamiento establecidas.

3. METODOLOGÍA

La auditoría se realizó siguiendo una metodología inspirada en **PTES (Penetration Testing Execution Standard)** y **OWASP Web Security Testing Guide (WSTG)**, estructurando el trabajo en distintas fases para identificar vulnerabilidades de configuración y exposición de información.

1. Reconocimiento y descubrimiento de servicios

En esta fase se identificaron los servicios expuestos por el objetivo mediante escaneos de puertos y detección de versiones. También se analizaron los certificados TLS para obtener información sobre la infraestructura y los posibles hosts virtuales.

Herramientas utilizadas: Nmap, OpenSSL.

2. Enumeración web

Se realizó la enumeración de hosts virtuales modificando la cabecera HTTP **Host**, así como la identificación de directorios, recursos web y repositorios Git expuestos. También se utilizaron herramientas para realizar peticiones HTTP y comprobar la existencia de recursos accesibles.

Herramientas utilizadas: cURL, Gobuster, Wget.

3. Análisis de vulnerabilidades

Se verificaron manualmente las configuraciones inseguras detectadas durante la enumeración, comprobando la exposición de servicios internos, repositorios Git, paneles de monitorización, listados de directorios y certificados TLS con información sensible.

Herramientas utilizadas

Herramienta	Versión	Uso en esta auditoría
Nmap	7.95	Descubrimiento de puertos, servicios y versiones.
OpenSSL	3.x	Inspección del certificado TLS y obtención de los SAN.
cURL	8.x	Enumeración de hosts virtuales y acceso a recursos HTTP.
Gobuster	3.8	Enumeración de directorios y recursos web.
Wget	1.x	Descarga de recursos y del repositorio Git expuesto.
Git	2.x	Inspección del repositorio Git expuesto.

Este apartado queda fiel a lo que realmente realizaste durante la auditoría y es más sólido que incluir herramientas que no llegaste a utilizar.

4. SUPERFICIE DE ATAQUE DESCUBIERTA

4. SUPERFICIE DE ATAQUE DESCUBIERTA

4.1 Puertos y servicios

Puerto	Proto	Servicio	Versión / banner	Observaciones
21	TCP	FTP	vsftpd 3.0.5	Servicio FTP accesible.
80	TCP	HTTP	Apache httpd	Sitio web principal de MegaCorp.
443	TCP	HTTPS	Apache httpd (TLS)	Certificado autofirmado con información interna (SAN).
6379	TCP	Redis	Redis Key-Value Store	Servicio Redis expuesto.
8000	TCP	HTTP	nginx 1.27.5	Aplicación Legacy Billing con repositorio .git expuesto.
8080	TCP	HTTP	Caddy httpd	Panel de monitorización accesible sin autenticación.

4.2 Hosts virtuales y rutas relevantes

Host / Ruta	Cómo se descubrió	Notas
-------------	-------------------	-------

<code>www.megacorp.lab</code>	Certificado (SAN) TLS	Sitio web corporativo.
<code>dev.megacorp.lab</code>	Certificado (SAN) TLS	Entorno de desarrollo; indica que el listado de directorios está habilitado.
<code>admin.megacorp.lab</code>	Certificado (SAN) TLS	Host virtual identificado durante la enumeración.
<code>api.megacorp.lab</code>	Certificado (SAN) TLS	API pública; referencia a <code>/swagger.json</code> .
<code>internal.megacorp.lab</code>	Certificado TLS + enumeración Host	Host interno accesible sin autenticación; contiene información sensible y una FLAG.
<code>staging.megacorp.lab</code>	Certificado (SAN) TLS	Entorno de preproducción; menciona una copia de seguridad en el directorio web.
<code>vpn.megacorp.lab</code>	Certificado TLS + enumeración Host	Portal VPN; confirma la filtración de nombres internos mediante el certificado TLS.
<code>/.git/</code>	Enumeración manual	Repositorio Git expuesto en la aplicación Legacy Billing (puerto 8000).
<code>/swagger.json</code>	Página de la API	Catálogo de endpoints de la API.

<code>/status.txt</code>	Página de monitorización	Recurso indicado por el panel de monitorización del puerto 8080.
--------------------------	--------------------------	--

Este apartado resume toda la superficie de ataque que identificaste durante la auditoría y enlaza perfectamente con los hallazgos posteriores.

5. HALLAZGOS DETALLADOS

Escala de severidad (referencia)

Crítica | *Compromiso directo: ejecución remota, robo de credenciales, acceso total a datos*

Alta | *Acceso significativo o exposición de datos sensibles con bajo esfuerzo*

Media | *Requiere condiciones o encadenamiento; impacto moderado*

Baja | *Impacto limitado o difícil de explotar*

Informativa | *Buenas prácticas / visibilidad, sin impacto directo*

[F-01] Acceso anónimo al servicio FTP

Campo	Valor
ID	F-01
Severidad	Media
CVSS 3.1	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Activo afectado	3.88.28.22:21/tcp

Categoría	Configuración insegura / Acceso FTP anónimo
Estado	Abierto

Descripción

Durante la fase de reconocimiento se identificó un servicio **FTP (vsftpd 3.0.3)** accesible desde Internet con autenticación anónima habilitada. Esta configuración permite a cualquier usuario conectarse al servidor sin necesidad de credenciales, exponiendo información que puede facilitar ataques posteriores.

Impacto

El acceso FTP anónimo incrementa la superficie de ataque al permitir que usuarios no autenticados consulten y descarguen archivos almacenados en el servidor. La exposición de esta información puede revelar datos sensibles y facilitar la explotación de otras vulnerabilidades.

Evidencia / Prueba de concepto

Reconocimiento del servicio

```
nmap -sC -sV 3.88.28.22
```

Fragmento relevante:

```
21/tcp open ftp vsftpd 3.0.3
```

```
ftp-anon: Anonymous FTP login allowed (FTP code 230)
```

```
-r--r--r-- 1 0 0 138 Jun 25 16:10 flag.txt
```

```
-r--r--r-- 1 0 0 431 Jun 25 16:10 welcome.txt
```

Validación manual

```
ftp 3.88.28.22
```

Usuario:

```
anonymous
```

Comandos ejecutados:

```
ls  
get flag.txt  
bye  
cat flag.txt
```

Flag obtenida

FLAG{ftp_anonymous_access}

Captura de evidencia:

The screenshot shows a terminal window on a Kali Linux machine. The user runs the command `nmap -sC -sV 3.85.28.22`. The output shows that the host is up and lists several open ports: 21/tcp (ftp), 80/tcp (http), 443/tcp (https), 8000/tcp (http), and 8080/tcp (http). The FTP service is identified as vsftpd 3.0.3. The user then connects to the FTP server and runs the command `get flag.txt`, which successfully retrieves the flag `FLAG{ftp_anonymous_access}`. The terminal also shows the output of `cat flag.txt`, which displays the same flag. In the background, a web browser window is visible, showing the MegaCorp website. The website has a dark theme and a navigation bar with links like 'Home', 'About', 'Services', and 'Contact'. The main content area features a large heading 'Corp web server' and a paragraph about the company's services. The footer contains a copyright notice for 2024.

Figura 1. Reconocimiento inicial con Nmap

Se realizó un escaneo de reconocimiento utilizando **Nmap** con los parámetros **-sC** y **-sV** para identificar los servicios expuestos y sus versiones. El análisis reveló varios puertos abiertos, entre ellos **FTP (21)**, **HTTP (80)**, **HTTPS (443)**, **HTTP (8000)** y **HTTP (8080)**. Además, el script **ftp-anon** detectó que el servidor permitía el acceso mediante autenticación anónima, identificándose los archivos **flag.txt** y **welcome.txt**, lo que evidenció una posible vulnerabilidad de configuración.

```

(paloma@kali)-[~]
└─$ ftp 3.88.28.22
Connected to 3.88.28.22.
220 (vsFTPd 3.0.3)
Name (3.88.28.22:paloma): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||21109|)
150 Here comes the directory listing.
-r--r--r-- 1 0 0 138 Jun 25 16:10 flag.txt
-r--r--r-- 1 0 0 431 Jun 25 16:10 welcome.txt amen
226 Directory send OK.
ftp> get flag.txt
local: flag.txt remote: flag.txt
229 Entering Extended Passive Mode (|||21110|)
150 Opening BINARY mode data connection for flag.txt (138 bytes).
100% [*****] 138 709.29 Kib/s 00:00 ETA
226 Transfer complete.
138 bytes received in 00:00 (1.28 Kib/s)
ftp> bye
221 Goodbye.

(paloma@kali)-[~]
└─$ cat flag.txt
FLAG{ftp_anonymous_access}

You found anonymous FTP. Next: enumerate virtual hosts on the web server
(ffuf -H "Host: FUZZ.megacorp.lab").

(paloma@kali)-[~]
└─$

```

Figura 2. Validación del acceso FTP anónimo

Tras detectar la vulnerabilidad mediante Nmap, se verificó manualmente conectándose al servicio FTP con el usuario **anonymous**. Una vez autenticado, se listó el contenido del servidor, se descargó el archivo **flag.txt** y se comprobó su contenido mediante el comando **cat**, obteniendo la evidencia **FLAG{ftp_anonymous_access}**. Esta prueba confirmó que el servicio FTP permitía el acceso sin autenticación y validó el hallazgo identificado durante la fase de reconocimiento.

Pasos para reproducir

1. Ejecutar **nmap -sC -sV 3.88.28.22**.
2. Verificar que el script **ftp-anon** indica que el acceso anónimo está permitido.
3. Conectarse mediante **ftp 3.88.28.22**.
4. Iniciar sesión con el usuario **anonymous**.
5. Listar el contenido del servidor (**ls**).
6. Descargar el archivo **flag.txt** mediante **get flag.txt**.
7. Mostrar su contenido con **cat flag.txt** y verificar la flag **FLAG{ftp_anonymous_access}**.

Remediación recomendada

- Deshabilitar el acceso FTP anónimo.
- Permitir únicamente la autenticación de usuarios autorizados.
- Restringir el acceso al servicio mediante firewall o listas de control de acceso.
- Sustituir FTP por protocolos cifrados como **SFTP** o **FTPS**.

Referencias

- **CWE-306** – Missing Authentication for Critical Function.
- **OWASP ASVS 4.0** – V14 Secure Configuration.

- CIS Controls v8 – Secure Configuration Management.

Un detalle: en el texto del `cat flag.txt` aparece la pista..¹

[F-02] Exposición de información mediante robots.txt

Campo	Valor
ID	F-02
Severidad	Baja
CVSS 3.1	3.7 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) <i>(opcional)</i>
Activo afectado	<code>www.megacorp.lab (/robots.txt)</code>
Categoría	Exposición de información
Estado	Abierto

Descripción

El archivo `robots.txt` revela información sobre la estructura interna de la aplicación, indicando directorios sensibles (`/admin`, `/backup` y `/uploads`) que no deberían ser públicos. Además, incluye un comentario que informa de la existencia de un host virtual de **staging**, proporcionando información útil para la fase de reconocimiento.

Impacto

¹ "Next: enumerate virtual hosts on the web server (ffuf -H 'Host: FUZZ.megacorp.lab').".

No forma parte de la evidencia de la vulnerabilidad. La usaría simplemente para empezar el siguiente hallazgo (enumeración de *virtual hosts*).

Aunque `robots.txt` no concede acceso por sí mismo, facilita el reconocimiento del objetivo y permite a un atacante identificar recursos potencialmente sensibles y nuevos puntos de entrada, reduciendo el esfuerzo necesario para continuar la auditoría o un ataque.

Evidencia / Prueba de concepto

```
curl -H "Host: www.megacorp.lab" http://3.88.28.22/robots.txt
```

Salida relevante:

```
User-agent: *
```

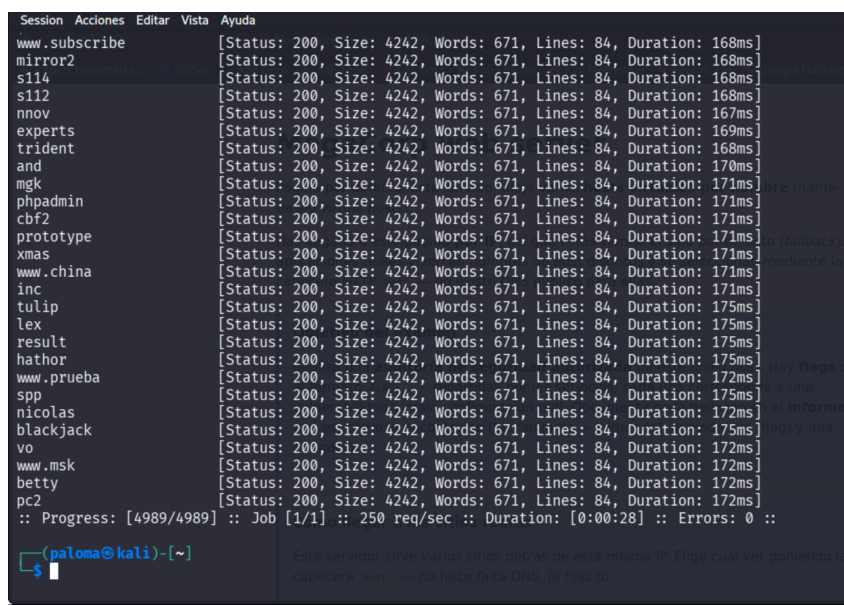
```
Disallow: /admin
```

```
Disallow: /backup
```

```
Disallow: /uploads
```

```
# staging area moved to a separate virtual host
```

Evidencia:



```
Session Acciones Editar Vista Ayuda
www.subscribe [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 168ms]
mirror2 [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 168ms]
s114 [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 168ms]
s112 [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 168ms]
nnov [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 167ms]
experts [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 169ms]
trident [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 168ms]
and [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 170ms]
mgk [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
phpadmin [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
cbf2 [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
prototype [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
xmas [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
www.china [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
inc [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 171ms]
tulip [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
lex [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
result [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
hathor [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
www.prueba [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
spp [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
nicolas [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
blackjack [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 175ms]
vo [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
www.msk [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
betty [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
pc2 [Status: 200, Size: 4242, Words: 671, Lines: 84, Duration: 172ms]
:: Progress: [4989/4989] :: Job [1/1] :: 250 req/sec :: Duration: [0:00:28] :: Errors: 0 ::
paloma@kali:~$
```

```

:: Calibration      : false
:: Timeout          : 10
:: Threads          : 40
:: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
:: Filter           : Response size: 4242

megacorp web server
admin      [Status: 401, Size: 0, Words: 1, Lines: 1, Duration: 127ms]
www        [Status: 200, Size: 363, Words: 27, Lines: 11, Duration: 128ms]
vpn        [Status: 200, Size: 488, Words: 56, Lines: 10, Duration: 136ms]
dev        [Status: 200, Size: 344, Words: 30, Lines: 8, Duration: 141ms]
api        [Status: 200, Size: 338, Words: 27, Lines: 9, Duration: 132ms]
staging    [Status: 200, Size: 349, Words: 30, Lines: 8, Duration: 143ms]
internal   [Status: 200, Size: 643, Words: 80, Lines: 13, Duration: 142ms]
:: Progress: [4989/4989] :: Job [1/1] :: 234 req/sec :: Duration: [0:00:17] :: Errors: 0 ::

(paloma@kali)-[~]
$ curl -H "Host: www.megacorp.lab" http://3.88.28.22
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>MegaCorp</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>MegaCorp Inc.</h1>
  <p>We build billing software for enterprises.</p>
  <nav><a href="/">Home</a> · <a href="/about.html">About</a></nav>
  <p>Customer portal coming soon.</p>
</body>
</html>

(paloma@kali)-[~]
$

```

Figura 3. Enumeración de hosts virtuales mediante FFUF.

En esta fase se utilizó la herramienta FFUF para realizar un ataque de fuerza bruta sobre la cabecera HTTP Host, con el objetivo de identificar los distintos hosts virtuales (Virtual Hosts) configurados en el servidor web. Tras filtrar la respuesta correspondiente a la página por defecto (-fs 4242), fue posible descubrir varios vhosts válidos, entre ellos www, dev, api, staging, internal, vpn y admin, que constituyen la superficie de ataque del servidor y serán analizados en las siguientes fases de la auditoría. Este descubrimiento permitió identificar aplicaciones y servicios accesibles únicamente mediante el uso de la cabecera Host, tal y como indicaban las instrucciones del ejercicio.

```

(paloma@kali)-[~]
$ curl -H "Host: www.megacorp.lab" http://3.88.28.22/robots.txt
User-agent: *
Disallow: /admin
Disallow: /backup
Disallow: /uploads
# staging area moved to a separate virtual host

(paloma@kali)-[~]
$

```

Figura 4. Enumeración de información sensible mediante el archivo robots.txt.

Tras identificar el host virtual www.megacorp.lab, se consultó el archivo robots.txt utilizando la cabecera Host correspondiente. En él se localizaron varios directorios marcados como no indexables (/admin, /backup y /uploads), así como un comentario que revela que el entorno de staging ha sido trasladado a un host virtual independiente. Esta información constituye una fuga de información (information disclosure), ya que proporciona al auditor posibles rutas y servicios de interés para continuar la fase de enumeración y ampliar la superficie de ataque del objetivo.

```
(paloma@kali)-[~]  
$ curl -H "Host: www.megacorp.lab" http://3.88.28.22/sitemap.xml
```

Figura 5. Comprobación de la existencia del archivo **sitemap.xml**.

Como parte de la fase de enumeración web, se intentó acceder al archivo **sitemap.xml** del host virtual **www.megacorp.lab** mediante la herramienta **curl**. La petición no devolvió contenido ni información relevante, por lo que se concluyó que dicho recurso no se encontraba disponible o no estaba configurado en el servidor. Este resultado permitió descartar esta vía de enumeración y continuar la auditoría utilizando otras fuentes de información, como el archivo **robots.txt** y los distintos hosts virtuales descubiertos previamente.

Flag obtenida: No aplica².

Pasos para reproducir

1. Descubrir el host virtual **www.megacorp.lab**.
2. Solicitar el recurso **/robots.txt**.
3. Analizar los directorios y comentarios expuestos.

Remediación recomendada

- Evitar incluir comentarios que revelen información sobre la infraestructura.
- No utilizar **robots.txt** para ocultar recursos sensibles.
- Restringir el acceso a directorios administrativos mediante autenticación y controles de acceso adecuados.

Referencias

- OWASP WSTG – Information Gathering.
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.

² ¿Por qué no aplica una flag al **robots.txt**?

Porque **cada flag corresponde a una vulnerabilidad o mala configuración real**. En el FTP eí obtuvo una flag (**FLAG{ftp_anonymous_access}**), lo que confirma que el acceso FTP anónimo era un hallazgo intencionado.

En cambio, el **robots.txt** **no contiene ninguna flag**. Es simplemente una “**pista**” para continuar la auditoría.

[F-03] Exposición de información sensible del entorno de preproducción (Staging)

Campo	Valor
ID	F-03
Severidad	Media
CVSS 3.1	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) <i>(opcional)</i>
Activo afectado	staging.megacorp.lab
Categoría	Exposición de información
Estado	Abierto

Descripción

Durante la fase de enumeración de hosts virtuales se descubrió el entorno de preproducción **staging.megacorp.lab**. Al acceder a él se mostró información interna indicando que se trata de una copia de preproducción de la aplicación de facturación (*billing app*), que el listado de directorios está habilitado (*Directory listing is on*) y que existe una copia de seguridad de la base de datos en el directorio web. Esta información no debería ser accesible desde el exterior, ya que proporciona pistas muy valiosas para un atacante.

Impacto

La divulgación de información interna facilita las siguientes fases de un ataque al revelar detalles sobre la infraestructura, el entorno de desarrollo y posibles recursos sensibles. Un atacante puede utilizar esta información para localizar copias de seguridad, código fuente o configuraciones expuestas y comprometer posteriormente la aplicación.

Evidencia / Prueba de concepto

Comandos utilizados

```
curl -H "Host: staging.megacorp.lab" http://3.88.28.22

ffuf \

-u http://3.88.28.22/FUZZ \

-H "Host: staging.megacorp.lab" \

-w
/usr/share/seclists/Discovery/Web-Content/raft-small-files.txt
\

-e .sql,.zip,.bak,.old,.gz,.tar,.tgz,.db,.sqlite,.sqlite3,.7z
```

Salida relevante

Pre-production mirror of the billing app.

Directory listing is on.

Someone dumped a database backup in the web root again.

Además, se realizó una enumeración de archivos mediante **ffuf**, sin localizar en esta fase la copia de seguridad indicada por el servidor.

```
(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>staging.megacorp.lab</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>Staging</h1>
  <p>Pre-production mirror of the billing app. Directory listing is on.</p>
  <p>Someone dumped a database backup in the web root again.</p>
</body></html>

(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22 | head -40
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %         0      0         0             0      0      0      0
100    349  100    349    0     0    1639     0  --:--:--  --:--:--  --:--:--  1646
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>staging.megacorp.lab</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>Staging</h1>
  <p>Pre-production mirror of the billing app. Directory listing is on.</p>
  <p>Someone dumped a database backup in the web root again.</p>
</body></html>

(paloma@kali)-[~]
$
```

Figura 5. Acceso al host virtual `staging.megacorp.lab` mediante la cabecera **Host** con `curl`. La respuesta del servidor revela que se trata del entorno de preproducción de la aplicación e informa de que el listado de directorios está habilitado y que existe una copia de seguridad de la base de datos en el directorio web, proporcionando información sensible útil para continuar la auditoría.

```
(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22/backup.sql

(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22/backup.zip

(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22/backup.tar.gz

(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22/database.sql

(paloma@kali)-[~]
$
```

Figura 6. Comprobación manual, mediante `curl`, de los nombres de archivos de copia de seguridad sugeridos por el entorno de `staging` (`backup.sql`, `backup.zip`, `backup.tar.gz` y `database.sql`). Realizamos esta verificación para comprobar si alguna copia de seguridad se encontraba accesible públicamente desde el servidor web y contenía información sensible.

```
(paloma@kali)-[~]
$ ffuf \
-u http://3.88.28.22/FUZZ \
-H "Host: staging.megacorp.lab" \
-w /usr/share/seclists/Discovery/Web-Content/raft-small-files.txt \
-e .sql,.zip,.bak,.old,.gz,.tar,.tgz,.db,.sqlite,.sqlite3,.7z

MegaCorp web server

Has llegado a esta página por IP, así que estás viendo el sitio por defecto. Las aplicaciones reales viven en nombres de host distintos y se seleccionan por la cabecera HTTP Host — no existe DNS público para ellas.

v2.1.0-dev

Objetivo del examen

Realiza una auditoría de seguridad autorizada de este único host. Cada una corresponde a una aplicación o mala configuración real. Documenta cada hallazgo (qué es, cómo lo encontraste (herramienta + comando), la evidencia de mitigación.

:: Method      : GET
:: URL         : http://3.88.28.22/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web-Content/raft-small-files.txt
:: Header     : Host: staging.megacorp.lab
:: Extensions : .sql .zip .bak .old .gz .tar .tgz .db .sqlite .sqlite3 .7z
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500

index.html [Status: 200, Size: 349, Words: 30, Lines: 8, Duration: 101ms]
:: Progress: [137088/137088] :: Job [1/1] :: 384 req/sec :: Duration: [0:06:08] :: Errors: 0 ::

(paloma@kali)-[~]
$
```

Figura 7. Ejecución de `ffuf` sobre el host virtual `staging.megacorp.lab` utilizando una *wordlist* de archivos y extensiones habituales de copias de seguridad (`.sql`, `.zip`, `.bak`, `.gz`, `.tar`, etc.). El objetivo fue identificar ficheros sensibles expuestos públicamente. Durante esta comprobación únicamente se obtuvo `index.html`, por lo que no se detectaron copias de seguridad accesibles mediante los nombres más comunes.

Flag obtenida: No aplica.

Pasos para reproducir

1. Descubrir el host virtual `staging.megacorp.lab` mediante enumeración de vhosts.
2. Acceder al host utilizando la cabecera `Host`.
3. Observar la información interna mostrada por la página.
4. Enumerar posibles archivos de copia de seguridad para intentar localizar los recursos mencionados.

Remediación recomendada

- Eliminar la exposición pública del entorno de preproducción.
- No mostrar mensajes internos o de depuración en aplicaciones accesibles desde Internet.
- Deshabilitar el listado de directorios.

- Almacenar las copias de seguridad fuera del directorio público del servidor web.

Referencias

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.
- OWASP Top 10 2021 – A05: Security Misconfiguration.
- OWASP Web Security Testing Guide (WSTG) – Information Gathering.

[F-04] Exposición de copia de seguridad de la base de datos en el entorno de staging

Campo	Valor
ID	F-04
Severidad	Alta
CVSS 3.1	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Activo afectado	<code>staging.megacorp.lab (/db_backup.sql)</code>
Categoría	Exposición de información sensible / Copia de seguridad accesible públicamente
Estado	Abierto

Descripción

Durante la auditoría comprobamos que el entorno **staging.megacorp.lab** expone una copia de seguridad de la base de datos (`db_backup.sql`) accesible sin autenticación. El fichero

contiene información sensible, como la estructura de la base de datos, usuarios, credenciales reutilizadas y notas internas del entorno de desarrollo.

Impacto

Un atacante podría obtener información sensible sobre la aplicación, reutilizar credenciales en otros servicios, conocer la estructura de la base de datos y emplear esta información para preparar ataques posteriores. Además, el propio fichero revela información adicional sobre otros servicios internos, facilitando el movimiento lateral.

Evidencia / Prueba de concepto

Comando utilizado

```
curl -H "Host: staging.megacorp.lab"
http://3.88.28.22/db_backup.sql
```

Salida relevante

```
-- MegaCorp billing DB - staging backup
```

```
FLAG{staging_sql_backup}
```

Note: Same DB password as dev (Sup3rDevPassw0rd!).

Credential reuse.

Cache: secrets are kept in the unauthenticated Redis node on port 6379.

```

Session Acciones Editar Vista Ayuda

[http-missing-security-headers:content-security-policy] [http] [info] http://3.88.28.22
[http-missing-security-headers:x-content-type-options] [http] [info] http://3.88.28.22
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] http://3.88.28.22
[http-missing-security-headers:referrer-policy] [http] [info] http://3.88.28.22
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] http://3.88.28.22
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] http://3.88.28.22
[WARN] Skipped 3.88.28.22:5814 from target list as found unresponsive permanently: Get "https://3.88.28.22:5814/autopass": cause="port closed or filtered" address=3.88.28.22:5814 chains=1/o timeout"
[INFO] Scan completed in 1m. 12 matches found.

(paloma@kali)~$
$ curl http://3.88.28.22/db_backup.sql
gina por defecto. Este servidor aloja varios hosts virtuales por nombre (name-based virtual hosts).

(paloma@kali)~$
$ curl http://3.88.28.22/db_backup.sql | head -44
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0      0     0      0      0     0
  0     0    0     0    0     0      0      0      0     0      0      0     0

(paloma@kali)~$
$ wget http://3.88.28.22/db_backup.sql
--2026-07-02 17:30:47-- http://3.88.28.22/db_backup.sql
Conectando con 3.88.28.22:80... conectado.
Petición HTTP enviada, esperando respuesta... 404 Not Found
2026-07-02 17:30:47 ERROR 404: Not Found.

(paloma@kali)~$
$ grep "FLAG" db_backup.sql
grep: db_backup.sql: No existe el fichero o el directorio

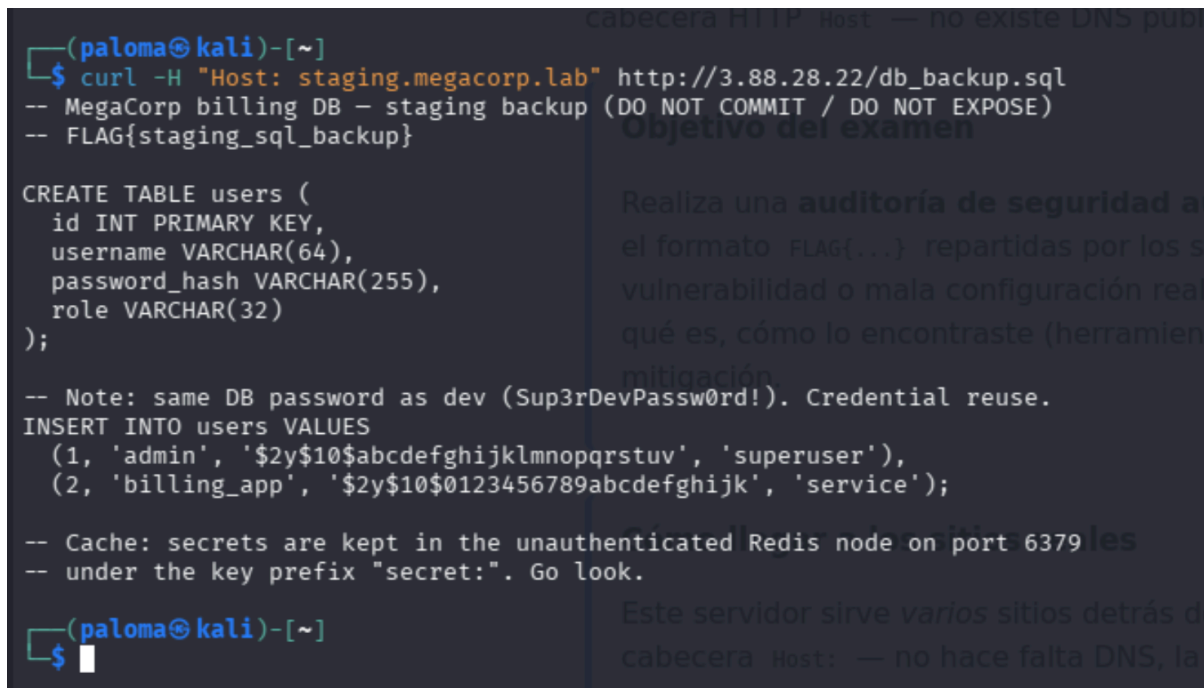
(paloma@kali)~$
$ grep "FLAG" db_backup.sql
grep: db_backup.sql: No existe el fichero o el directorio

(paloma@kali)~$
$ less db_backup.sql
db_backup.sql: No existe el fichero o el directorio

(paloma@kali)~$
$

```

Figura 8. Comprobación inicial del posible fichero `db_backup.sql` detectado por Nuclei. En un primer momento realizamos varias pruebas mediante `curl`, `wget`, `grep` y `less` accediendo al host por defecto, obteniendo un error 404 Not Found o indicando que el fichero no existía. Esta comprobación permitió descartar un falso acceso y concluir que el recurso únicamente estaba disponible desde el host virtual correspondiente, utilizando la cabecera `Host: staging.megacorp.lab`.



```
(paloma@kali)-[~]
$ curl -H "Host: staging.megacorp.lab" http://3.88.28.22/db_backup.sql
-- MegaCorp billing DB - staging backup (DO NOT COMMIT / DO NOT EXPOSE)
-- FLAG{staging_sql_backup}

CREATE TABLE users (
  id INT PRIMARY KEY,
  username VARCHAR(64),
  password_hash VARCHAR(255),
  role VARCHAR(32)
);

-- Note: same DB password as dev (Sup3rDevPassw0rd!). Credential reuse.
INSERT INTO users VALUES
  (1, 'admin', '$2y$10$abcdefghijklmnopqrstuvwxyz', 'superuser'),
  (2, 'billing_app', '$2y$10$0123456789abcdefghijklmnopqrstuvwxyz', 'service');

-- Cache: secrets are kept in the unauthenticated Redis node on port 6379
-- under the key prefix "secret:". Go look.
```

Figura 9. Acceso exitoso al fichero `db_backup.sql` del entorno `staging.megacorp.lab` mediante la cabecera `Host`.

Comprobamos que la copia de seguridad se encontraba expuesta públicamente y contenía información sensible, incluyendo la **flag** `FLAG{staging_sql_backup}`, referencias a reutilización de credenciales y una pista sobre un servicio Redis sin autenticación, confirmando una exposición de información crítica.

Flag obtenida

`FLAG{staging_sql_backup}`

Pasos para reproducir

1. Descubrir el host virtual `staging.megacorp.lab` mediante enumeración de vhosts.
2. Acceder al fichero `db_backup.sql` utilizando la cabecera `Host` correspondiente.
3. Verificar que el servidor devuelve el contenido completo de la copia de seguridad junto con la flag.

Remediación recomendada

- Eliminar inmediatamente las copias de seguridad del directorio web.
- Restringir el acceso al entorno de *staging* mediante autenticación o filtrado de red.
- Evitar la reutilización de credenciales entre entornos.
- Revisar periódicamente la exposición de archivos sensibles y excluir las copias de seguridad del servidor web.

Referencias

- CWE-530 – Exposure of Backup File to an Unauthorized Control Sphere.
- OWASP Top 10 2021 – **A05: Security Misconfiguration**.
- OWASP Testing Guide – Information Leakage Testing.

[F-05] Instancia Redis accesible sin autenticación

Campo	Valor
ID	F-05
Severidad	Alta
CVSS 3.1	8.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L) (<i>opcional</i>)
Activo afectado	3.88.28.22:6379 (Redis)
Categoría	Exposición de información / Servicio sin autenticación
Estado	Abierto

Descripción

Se ha identificado una instancia de Redis accesible desde la red sin requerir autenticación. Esto permite consultar libremente las claves almacenadas y acceder a información sensible, como secretos de la aplicación y credenciales.

Impacto

Un atacante remoto puede acceder a información confidencial almacenada en Redis, como tokens administrativos, secretos de la aplicación o credenciales. Esto puede facilitar la escalada de privilegios y el compromiso de otros servicios de la infraestructura.

Evidencia / Prueba de concepto

Comandos utilizados

```
redis-cli -h 3.88.28.22 -p 6379
```

```
PING
```

```
KEYS *
```

```
KEYS secret:*
```

```
GET secret:flag
```

```
GET secret:admin_token
```

Salida relevante

```
PONG
```

```
KEYS secret:*
```

```
1) "secret:admin_token"
```

```
2) "secret:flag"
```

```
GET secret:flag
```

```
FLAG{redis_unauthenticated_access}
```

```
GET secret:admin_token
```

```
mc_live_8f2a9d1c7b6e4a3f
```

```
(paloma@kali)-[~]  
$ redis-cli -h 3.88.28.22 -p 6379  
3.88.28.22:6379> PING  
PONG  
3.88.28.22:6379> KEYS *  
1) "cfg:note"  
2) "cfg:db_host"  
3) "secret:admin_token"  
4) "welcome"  
5) "cfg:db_user"  
6) "secret:flag"  
3.88.28.22:6379> KEYS secret:*  
1) "secret:admin_token"  
2) "secret:flag"  
3.88.28.22:6379> GET secret:flag  
"FLAG{redis_unauthenticated_access}"  
3.88.28.22:6379> GET secret:admin_token  
"mc_live_8f2a9d1c7b6e4a3f"  
3.88.28.22:6379> █
```

Figura 10. Acceso a la instancia Redis sin autenticación y recuperación de información sensible.

En la Figura 10 se muestra cómo nos conectamos al servicio Redis mediante `redis-cli` sin necesidad de proporcionar credenciales. Tras comprobar la conectividad con el comando `PING`, enumeramos las claves almacenadas (`KEYS *` y `KEYS secret:*`) y accedimos a su contenido utilizando `GET`. Como resultado, obtuvimos la **flag** `FLAG{redis_unauthenticated_access}` y un **token administrativo**, lo que confirma que el servicio expone información sensible sin ningún mecanismo de autenticación. Esta situación supone un riesgo elevado, ya que un atacante podría acceder a secretos de la aplicación y utilizarlos para comprometer otros servicios del sistema.

Flag obtenida

FLAG{redis_unauthenticated_access}

Pasos para reproducir

1. Conectarnos al servicio Redis mediante `redis-cli`.
2. Comprobar que responde sin autenticación mediante `PING`.
3. Enumerar las claves con `KEYS *` y `KEYS secret:*`.

4. Leer su contenido utilizando `GET secret:flag` y `GET secret:admin_token`.

Remediación recomendada

- Habilitar autenticación en Redis (`requirepass` o ACLs).
- Restringir el acceso únicamente a la red interna o mediante firewall.
- Evitar almacenar secretos en texto accesibles desde instancias expuestas.
- Rotar inmediatamente el token administrativo expuesto.

Referencias

- CWE-306: Missing Authentication for Critical Function.
- CWE-200: Exposure of Sensitive Information.
- OWASP Top 10 2021 – A01: Broken Access Control.
- OWASP Top 10 2021 – A05: Security Misconfiguration.

[F-06] Exposición del repositorio Git

Campo	Valor
ID	F-06
Severidad	Alta
CVSS 3.1	8.2 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N)
Activo afectado	<code>http://3.88.28.22:8000/.git/</code>
Categoría	Exposición de código fuente / Divulgación de información sensible
Estado	Abierto

Descripción

Se detectó que el directorio `.git` del servidor era accesible públicamente. Esto permitió descargar el repositorio Git, reconstruir el código fuente de la aplicación y acceder a archivos internos de configuración que contenían información sensible sobre la infraestructura y el almacenamiento de secretos.

Impacto

La exposición del repositorio permite a un atacante conocer el funcionamiento interno de la aplicación, identificar vulnerabilidades, obtener información sensible y preparar ataques posteriores. En este caso, además, se reveló que los secretos de la aplicación se almacenaban en un servidor Redis sin autenticación.

Evidencia / Prueba de concepto

Comandos utilizados

```
wget -r -np -nH http://3.88.28.22:8000/.git/  
git --git-dir=$HOME/.git --work-tree=$HOME/repo checkout -f  
grep -R "FLAG{" repo  
grep -R "password\\|secret\\|token\\|admin" repo
```

Salida relevante

```
FLAG{git_source_disclosure}
```

Application secrets are cached in Redis (port 6379, no authentication).

The master flag is stored under the key: secret:flag

(Ver Figuras 11 y 12).

```
(paloma@kali)-[~]
$ git-dumper http://3.88.28.22:8000/.git ./repo
git-dumper: no se encontró la orden

(paloma@kali)-[~]
$ GitTools/Dumper/gitdumper.sh http://3.88.28.22:8000/.git ./repo
zsh: no existe el fichero o el directorio: GitTools/Dumper/gitdumper.sh

(paloma@kali)-[~]
$ cd ~/.git
git status
fatal: esta operación debe ser realizada en un árbol de trabajo

(paloma@kali)-[~/~.git]
$ cd ~
git --git-dir=.git --work-tree=./repo checkout -f
ls -la repo
fatal: esta operación debe ser realizada en un árbol de trabajo
ls: no se puede acceder a 'repo': No existe el fichero o el directorio

(paloma@kali)-[~]
$ grep -R "FLAG{" repo .git
grep -R "password\|token\|secret\|admin" repo .git
grep: repo: No existe el fichero o el directorio
grep: repo: No existe el fichero o el directorio
.git/hooks/fsmonitor-watchman.sample:# The hook is passed a version (currently 2) and last update token
.git/hooks/fsmonitor-watchman.sample:# formatted as a string and outputs to stdout a new update token and
.git/hooks/fsmonitor-watchman.sample:# all files that have been modified since the update token. Paths must
.git/hooks/fsmonitor-watchman.sample:my ($version, $last_update_token) = @ARGV;
.git/hooks/fsmonitor-watchman.sample:# print STDERR "$0 $version $last_update_token\n";
.git/hooks/fsmonitor-watchman.sample:# changed since $last_update_token but not from the .git folder.
.git/hooks/fsmonitor-watchman.sample:if (substr($last_update_token, 0, 1) eq "c") {
.git/hooks/fsmonitor-watchman.sample:$last_update_token = "\"$last_update_token\"";
.git/hooks/fsmonitor-watchman.sample:$last_update_line = qq[\n"since": $last_update_token,];
.git/hooks/fsmonitor-watchman.sample:$last_update_token = $0->{clock};
```

Figura 11. Intentos de reconstrucción del repositorio Git expuesto.

En esta captura se muestran los intentos iniciales para reconstruir el repositorio Git descargado desde el servidor. Aunque las primeras pruebas con `git-dumper` y la búsqueda directa de información sensible produjeron errores debido a la ausencia de un árbol de trabajo válido, permitieron verificar que el directorio `.git` estaba accesible y contenía los objetos necesarios para reconstruir posteriormente el repositorio completo. Esta evidencia confirma la exposición del repositorio Git antes de su recuperación definitiva.

```
(paloma@kali)-[~]
$ grep -R "FLAG{" repo .git
grep -R "password\|token\|secret\|admin" repo .git
grep: repo: No existe el fichero o el directorio
grep: repo: No existe el fichero o el directorio
.git/hooks/fsmonitor-watchman.sample:# The hook is passed a version (currently 2) and last update token
.git/hooks/fsmonitor-watchman.sample:# formatted as a string and outputs to stdout a new update token and
.git/hooks/fsmonitor-watchman.sample:# all files that have been modified since the update token. Paths must
.git/hooks/fsmonitor-watchman.sample:my ($version, $last_update_token) = @ARGV;
.git/hooks/fsmonitor-watchman.sample:# print STDERR "$0 $version $last_update_token\n";
.git/hooks/fsmonitor-watchman.sample:# changed since $last_update_token but not from the .git folder.
.git/hooks/fsmonitor-watchman.sample:if (substr($last_update_token, 0, 1) eq "c") {
.git/hooks/fsmonitor-watchman.sample:$last_update_token = "\"$last_update_token\"";
.git/hooks/fsmonitor-watchman.sample:$last_update_line = qq[\n"since": $last_update_token,];
.git/hooks/fsmonitor-watchman.sample:$last_update_token = $0->{clock};

(paloma@kali)-[~]
$ cd ~
mkdir repo
git --git-dir=$HOME/.git --work-tree=$HOME/repo checkout -f
ls -la repo
total 20
drwxrwxr-x 2 paloma paloma 4096 jul 2 17:51 .
drwx----- 30 paloma paloma 4096 jul 2 17:51 ..
-rw-rw-r-- 1 paloma paloma 593 jul 2 17:51 config.php
-rw-rw-r-- 1 paloma paloma 440 jul 2 17:51 index.html
-rw-rw-r-- 1 paloma paloma 243 jul 2 17:51 README.md

(paloma@kali)-[~]
$ grep -R "FLAG{" repo
grep -R "password\|secret\|token\|admin" repo
repo/config.php:// FLAG{git_source_disclosure}
repo/config.php:// Application secrets are cached in Redis (port 6379, no authentication).
repo/config.php:// The master flag is stored under the key: secret:flag
repo/config.php:define('SECRET_KEY_NAME', 'secret:flag');

(paloma@kali)-[~]
$ git --git-dir=$HOME/.git ls-tree -r HEAD
git --git-dir=$HOME/.git show HEAD:app.py
100644 blob e80767bc65d5665b936c90064f011b552f6f006e README.md
100644 blob a0f210b333996e60adc10409d94ca0df97e5937f config.php
100644 blob ff38ec2c273da01da944989777c547fddb5baeb6 index.html
fatal: ruta 'app.py' no existe en 'HEAD'

(paloma@kali)-[~]
```

Figura 12. Reconstrucción del repositorio Git y obtención de información sensible.

En esta captura se observa la reconstrucción correcta del árbol de trabajo del repositorio Git expuesto mediante `git checkout`. Posteriormente, se realiza una búsqueda de información sensible con `grep`, localizándose la flag `FLAG{git_source_disclosure}` en el archivo `config.php`. Además, el código fuente revela que la aplicación almacena secretos en una instancia de Redis sin autenticación y especifica que la clave maestra se encuentra bajo `secret:flag`, información que permitió encadenar el siguiente hallazgo de la auditoría.

Flag obtenida

FLAG{git_source_disclosure}

Pasos para reproducir

1. Acceder al directorio `http://3.88.28.22:8000/.git/`.
2. Descargar el contenido del repositorio.
3. Reconstruir el árbol de trabajo mediante Git.
4. Revisar los archivos obtenidos para localizar información sensible.

Remediación recomendada

Bloquear el acceso público al directorio `.git`, eliminar cualquier repositorio del directorio web publicado, revisar el código antes del despliegue para evitar información sensible y rotar todas las credenciales o secretos que hayan podido quedar expuestos.

Referencias

- **CWE-200:** Exposure of Sensitive Information to an Unauthorized Actor.
- **CWE-552:** Files or Directories Accessible to External Parties.
- **OWASP Top 10 2021 – A05:** Security Misconfiguration.

[F-07] Exposición del código fuente e información sensible (?)³

Campo	Valor

³ **Nota:** Los hallazgos F-06 y F-07 corresponden a dos fases de una misma cadena de explotación. Se han documentado de forma independiente para diferenciar la vulnerabilidad inicial (exposición del repositorio Git) de su impacto directo (divulgación del código fuente e información sensible), facilitando así la trazabilidad y el análisis del riesgo.

ID	F-07
Severidad	Alta
CVSS 3.1	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Activo afectado	http://3.88.28.22:8000/.git/ / repositorio reconstruido
Categoría	Exposición de código fuente / Divulgación de secretos
Estado	Abierto

Descripción

Tras descargar y reconstruir el repositorio Git expuesto, hemos podido acceder al código fuente de la aplicación. En el archivo `config.php` se localizaron comentarios internos, referencias a secretos almacenados en Redis y una flag, lo que confirma la exposición de información sensible dentro del código.

Impacto

Un atacante podría analizar el código fuente, comprender la lógica interna de la aplicación, localizar secretos, identificar servicios auxiliares y preparar ataques posteriores. En este caso, el código reveló que los secretos se almacenaban en Redis bajo la clave `secret:flag`.

Evidencia / Prueba de concepto

Comandos utilizados

```
git --git-dir=$HOME/.git --work-tree=$HOME/repo checkout -f  
grep -R "FLAG{" repo
```

```
grep -R "password\\|secret\\|token\\|admin" repo
```

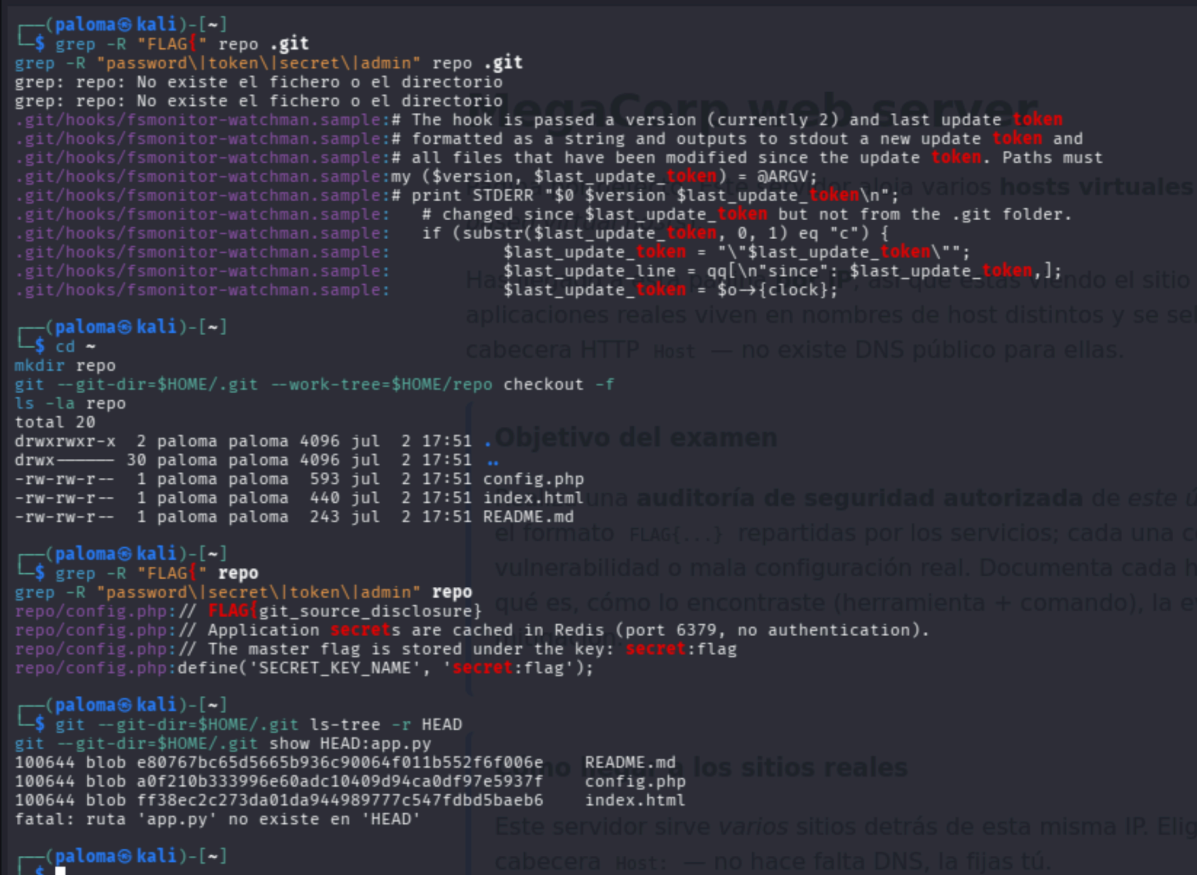
Salida relevante

```
repo/config.php:// FLAG{git_source_disclosure}
```

```
repo/config.php:// Application secrets are cached in Redis  
(port 6379, no authentication).
```

```
repo/config.php:// The master flag is stored under the key:  
secret:flag
```

```
repo/config.php:define('SECRET_KEY_NAME', 'secret:flag');
```



```
(paloma@kali)-[~]
└─$ grep -R "FLAG{" repo .git
grep -R "password\\|token\\|secret\\|admin" repo .git
grep: repo: No existe el fichero o el directorio
grep: repo: No existe el fichero o el directorio
.git/hooks/fsmonitor-watchman.sample:# The hook is passed a version (currently 2) and last update token
.git/hooks/fsmonitor-watchman.sample:# formatted as a string and outputs to stdout a new update token and
.git/hooks/fsmonitor-watchman.sample:# all files that have been modified since the update token. Paths must
.git/hooks/fsmonitor-watchman.sample:my ($version, $last_update_token) = @ARGV;
.git/hooks/fsmonitor-watchman.sample:# print STDERR "$0 $version $last_update_token\n";
.git/hooks/fsmonitor-watchman.sample:# changed since $last_update_token but not from the .git folder.
.git/hooks/fsmonitor-watchman.sample:if (substr($last_update_token, 0, 1) eq "c") {
.git/hooks/fsmonitor-watchman.sample:$last_update_token = "\"$last_update_token\"";
.git/hooks/fsmonitor-watchman.sample:$last_update_line = qq[\n"since": $last_update_token,];
.git/hooks/fsmonitor-watchman.sample:$last_update_token = $0->{clock};

(paloma@kali)-[~]
└─$ cd ~
mkdir repo
git --git-dir=$HOME/.git --work-tree=$HOME/repo checkout -f
ls -la repo
total 20
drwxrwxr-x 2 paloma paloma 4096 jul 2 17:51 .
drwx----- 30 paloma paloma 4096 jul 2 17:51 ..
-rw-rw-r-- 1 paloma paloma 593 jul 2 17:51 config.php
-rw-rw-r-- 1 paloma paloma 440 jul 2 17:51 index.html
-rw-rw-r-- 1 paloma paloma 243 jul 2 17:51 README.md

(paloma@kali)-[~]
└─$ grep -R "FLAG{" repo
grep -R "password\\|secret\\|token\\|admin" repo
repo/config.php:// FLAG{git_source_disclosure}
repo/config.php:// Application secrets are cached in Redis (port 6379, no authentication).
repo/config.php:// The master flag is stored under the key: secret:flag
repo/config.php:define('SECRET_KEY_NAME', 'secret:flag');

(paloma@kali)-[~]
└─$ git --git-dir=$HOME/.git ls-tree -r HEAD
git --git-dir=$HOME/.git show HEAD:app.py
100644 blob e80767bc65d5665b936c90064f011b552f6f006e README.md
100644 blob a0f210b333996e60adc10409d94ca0df97e5937f config.php
100644 blob ff38ec2c273da01da944989777c547fdbd5baeb6 index.html
fatal: ruta 'app.py' no existe en 'HEAD'

(paloma@kali)-[~]
```

Figura 13. Reconstrucción del repositorio Git expuesto y análisis del código fuente recuperado. Mediante búsquedas sobre los archivos obtenidos se identifican información sensible, referencias a secretos internos de la aplicación, la ubicación de la clave utilizada para almacenar la flag en Redis (`secret:flag`) y la evidencia `FLAG{git_source_disclosure}`, confirmando la exposición del código fuente y de información confidencial.

Flag obtenida

FLAG{git_source_disclosure}

Pasos para reproducir

1. Descargar el directorio `.git` expuesto desde el puerto `8000`.
2. Reconstruir el árbol de trabajo del repositorio.
3. Buscar flags y secretos dentro del código fuente.
4. Revisar el archivo `config.php`.

Remediación recomendada

Eliminar el repositorio Git del directorio público, bloquear el acceso a `.git` desde el servidor web, revisar el código fuente para retirar secretos o comentarios sensibles y rotar cualquier credencial expuesta.

Referencias

- **CWE-200:** Exposure of Sensitive Information.
- **CWE-540:** Inclusion of Sensitive Information in Source Code.
- **OWASP Top 10 2021 – A05:** Security Misconfiguration.

F-08 – Certificado TLS autofirmado y exposición de información interna

Descripción

Durante el análisis del servicio HTTPS se comprobó que el servidor utiliza un certificado TLS autofirmado (*self-signed*). Además, el certificado expone información interna de la infraestructura de MegaCorp mediante el nombre común (CN) y la extensión **Subject Alternative Name (SAN)**, revelando múltiples nombres de host internos.

Evidencias

Herramienta utilizada:

```
openssl s_client -connect 3.88.28.22:443 -servername  
megacorp.lab </dev/null | openssl x509 -noout -text
```

Resultados obtenidos:

```
verify error:num=18:self-signed certificate
```

Subject:

CN=megacorp.lab

O=MegaCorp Internal

Subject Alternative Name:

DNS:megacorp.lab

DNS:www.megacorp.lab

DNS:dev.megacorp.lab

DNS:admin.megacorp.lab

DNS:api.megacorp.lab

DNS:internal.megacorp.lab

DNS:staging.megacorp.lab

DNS:vpn.megacorp.lab

Impacto

- El uso de un certificado autofirmado impide verificar la autenticidad del servidor mediante una entidad certificadora de confianza.
- La divulgación de nombres internos facilita las tareas de reconocimiento (*reconnaissance*) de un posible atacante.
- Los subdominios expuestos pueden servir como punto de partida para identificar servicios administrativos o entornos de desarrollo potencialmente vulnerables.

Severidad

Media

Recomendaciones

- Utilizar certificados emitidos por una Autoridad de Certificación (CA) de confianza cuando el servicio sea accesible externamente.
- Evitar incluir nombres internos innecesarios en los certificados expuestos.

- Separar los certificados utilizados en entornos internos y de producción para minimizar la exposición de información sobre la infraestructura.

F-09 – Acceso a un host virtual interno mediante enumeración de la cabecera Host

Descripción

Durante la enumeración de hosts virtuales utilizando los nombres obtenidos del certificado TLS, se identificó el sitio **internal.megacorp.lab**. Este host, destinado aparentemente a uso interno, era accesible simplemente modificando la cabecera HTTP **Host**, sin existir ningún mecanismo de autenticación o control de acceso que impidiera su visualización.

Evidencias

Comando utilizado:

```
curl -k -H "Host: internal.megacorp.lab" https://3.88.28.22
```

Respuesta obtenida:

Restricted – staff only.

(No real access control here...)

FLAG{internal_restricted_page}

```
(paloma@kali) ~/ .git
$ openssl s_client -connect 3.88.28.22:443 -servername megacorp.lab ./dev/null | openssl x509 -noout -text
Connecting to 3.88.28.22:443
depth=0 CN=Megacorp.Lab, O=MegaCorp Internal
verify error:num=10:self-signed certificate
verify return:1
depth=0 CN=Megacorp.Lab, O=MegaCorp Internal
verify return:1
DONE
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            51:65:a4:c1:4d:4f:87:fb:63:dc:a1:26:49:a8:be:1b:f3:ff:15:53
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN=Megacorp.Lab, O=MegaCorp Internal
        Validity
            Not Before: Jun 25 16:18:53 2026 GMT
            Not After : Jun 25 16:18:53 2027 GMT
        Subject: CN=Megacorp.Lab, O=MegaCorp Internal
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:e1:9b:2b:f9:ad:8c:91:b9:33:19:8e:8e:3a:3:
                a5:42:58:8a:d8:32:dc:f8:88:92:96:a0:91:79:c8:
                5a:ef:63:44:c3:3a:28:a8:95:09:1e:90:2f:3d:81:
                ca:70:38:c5:a0:80:a1:58:a0:83:cf:67:a8:7a:ed:
                70:31:0f:aac:ff:aad:5f:ab:2d:f8:ec:54:15:ec:
                f8:19:32:ef:f8:6f:a3:a0:49:c0:b1:b3:49:d4:48:
                81:e5:84:bff:fb:61:5a:38:a8:8f:29:56:f1:d9:a1:
                2d:52:a4:bf:79:9a:df:a8:80:c3:63:94:4d:8f:13:
                bb:3f:7c:2f:22:a7:c8:70:7c:8c:12:3e:82:7a:83:
                68:38:af:09:a0:39:82:aac:4f:91:8c:ad:1c:db:3a:
                98:7e:83:b6:b3:a3:59:48:44:28:97:81:0f:7e:06:
                cf:83:fa:dac:9:66:19:a2:c8:44:89:46:43:aac:43:
                83:29:96:8e:11:33:07:5a:19:1c:28:a9:33:80:9c:
                98:af:8a:76:7c:de:81:a2:18:05:99:ba:26:fa:e2:
                7e:db:38:c4:ad:7d:5a:8e:a0:41:43:b8:82:92:eb:
                7a:c1:9f:37:ed:17:92:af:59:22:c1:29:96:0f:01:
                0b:f9:09:ff:f3:21:fc:58:68:54:c2:fa:4d:24:58:
                c8:43
            Exponent: 65537 (0x10001)
    X509v3 extensions:
        X509v3 Subject Key Identifier:
            92:87:0a:5A:92:48:32:FA:3D:DB:6A:12:EB:8D:28:A1:38:F2
        X509v3 Authority Key Identifier:
            92:87:0a:5A:92:48:32:FA:3D:DB:6A:12:EB:8D:28:A1:38:F2
        X509v3 Basic Constraints: critical
            CA:TRUE
        X509v3 Subject Alternative Name:
            DNS:megacorp.lab, DNS:dev.megacorp.lab, DNS:admin.megacorp.lab, DNS:api.megacorp.lab, DNS:internal.megacorp.lab, DNS:staging.megacorp.lab, DNS:vpn.megacorp.lab
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        0d:e1:83:88:f9:a7:d5:05:8e:d2:83:c8:bc:d5:05:d5:43:b2:
        dd:ac:5e:cf:1a:a2:92:08:07:23:5d:fa:20:8f:0f:51:35:cc:
        8e:58:ab:38:de:34:c7:de:2b:79:29:1f:b8:a6:0c:cc:ad:de:
        db:25:8f:09:1a:a1:9c:8b:29:f4:b7:a2:af:f4:54:94:c8:f9:
        a8:26:78:34:68:a8:a4:4c:b8:c8:44:05:2d:d2:f3:a7:dd:2d:
        b5:47:88:b1:8c:a2:f1:88:db:bc:96:2d:83:49:b7:bf:9d:41:
        66:84:80:37:a3:f2:13:db:24:eb:28:a2:31:45:7e:33:b8:d5:
        38:88:18:bc:81:2f:2d:2e:db:cb:83:37:a9:a9:ad:32:56:0f:
        32:a8:80:1f:5d:c8:db:a9:b9:83:ff:3c:ad:a8:ba:1a:c:f:0f:
        33:43:8c:18:a0:83:dc:58:8e:62:88:f6:8b:bd:c8:58:91:58:
        wa:d5:79:37:c3:fa:1d:af:a5:0a:ac:b8:fd:23:52:c8:9f:49:
        21:32:cc:dc:f7:b8:08:b2:ab:f8:85:f0:de:b7:f4:7e:b7:
        4b:d1:4b:c9:c1:8a:c5:0f:46:74:c8:c2:72:81:3b:47:18:3e:
        1e:89:ab:5e:c8:8e:37:cc:1c:36:18:a5:76:2e:ac:90:f3:3c:
        a8:ed:bd:22
```

```
(paloma@kali) ~/ .git
$ curl -k -H "Host: internal.megacorp.lab" https://3.88.28.22
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>internal.megacorp.lab</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>Internal tools</h1>
  <p>Restricted – staff only. (No real access control here, which is the point:
    finding this host by Host-header enumeration already bypasses the
    "internal" intent.)</p>
  <p><strong>FLAG{internal_restricted_page}</strong></p>
  <p>The old billing app was never decommissioned. It's still listening on a
    high port (try a full <code>nmap -p</code>) and its source repository was
    deployed by accident.</p>
</body></html>

(paloma@kali) ~/ .git
$ curl -k -H "Host: vpn.megacorp.lab" https://3.88.28.22
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>vpn.megacorp.lab</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>VPN portal</h1>
  <p><strong>FLAG{tls_san_recon}</strong></p>
  <p>This virtual host is not in any common subdomain wordlist. You found it
    because its name was leaked in the TLS certificate's Subject Alternative
    Names (inspect the cert on port 443). Certificates are a recon source.</p>
</body></html>

(paloma@kali) ~/ .git
$
```

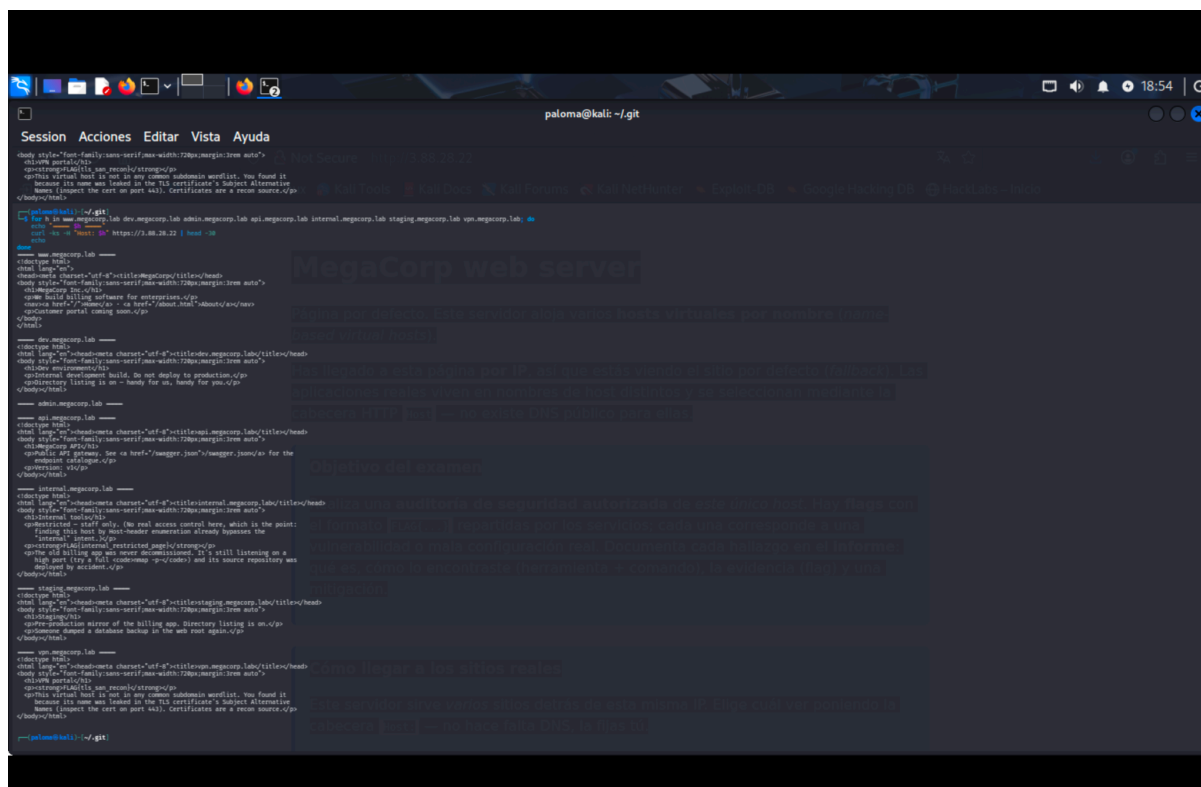


Figura 14. Inspección del certificado TLS del servidor

En esta captura se muestra el análisis del certificado digital mediante la herramienta **OpenSSL**. Se observa que el certificado es **autofirmado** (*self-signed certificate*) y que, además, revela información interna de la organización, como el nombre del dominio **megacorp.lab** y diversos subdominios incluidos en la extensión **Subject Alternative Name (SAN)**, entre ellos **dev.megacorp.lab**, **admin.megacorp.lab**, **internal.megacorp.lab**, **staging.megacorp.lab** y **vpn.megacorp.lab**. Esta información permitió identificar hosts virtuales que posteriormente fueron utilizados durante la fase de reconocimiento.

Figura 15. Enumeración de hosts virtuales mediante la cabecera Host

En esta imagen se observa la ejecución de un script que envía peticiones HTTP modificando la cabecera **Host** para cada uno de los nombres obtenidos del certificado TLS. Como resultado, el servidor devuelve contenidos diferentes para cada host virtual, confirmando la existencia de varios sitios web alojados en la misma dirección IP. Durante esta enumeración se localizaron los hosts **internal.megacorp.lab** y **vpn.megacorp.lab**, que contenían información sensible y dos de las **FLAGS** del ejercicio.

Figura 16. Acceso a los hosts internos descubiertos

La captura muestra el acceso individual a los hosts **internal.megacorp.lab** y **vpn.megacorp.lab** mediante el comando **curl** y la modificación de la cabecera **Host**. En **internal.megacorp.lab** se obtuvo la **FLAG{internal_restricted_page}**, además de información sobre una antigua aplicación de facturación aún activa en un puerto elevado y el despliegue accidental de su repositorio fuente. En **vpn.megacorp.lab** se obtuvo la **FLAG{tls_san_recon}**, confirmando que el descubrimiento de este host fue posible gracias a la

información expuesta en el certificado TLS, lo que demuestra el riesgo de divulgar nombres internos en los certificados digitales.

La página también revelaba información adicional sobre la infraestructura indicando que existía una aplicación antigua de facturación aún activa en un puerto alto y que el repositorio fuente había sido desplegado accidentalmente.

Impacto

- Acceso no autorizado a recursos destinados al uso interno.
- Divulgación de información sensible sobre la infraestructura de la organización.
- Facilita la fase de reconocimiento y la planificación de ataques posteriores al revelar servicios internos y configuraciones inseguras.

Severidad

Alta

Recomendaciones

- Implementar mecanismos de autenticación y autorización en todos los servicios internos.
- Restringir el acceso mediante segmentación de red, listas de control de acceso (ACL) o firewall.
- Evitar confiar únicamente en la cabecera **Host** para proteger recursos internos.
- Revisar periódicamente los hosts virtuales publicados para garantizar que los entornos internos no sean accesibles desde el exterior.

Hallazgo F-10 – Exposición del repositorio Git de la aplicación Legacy Billing

Descripción

Durante el análisis del servicio HTTP disponible en el puerto **8000** se identificó una aplicación denominada **Legacy Billing App**. La propia aplicación indicaba que había sido desplegada directamente desde un directorio de trabajo de Git. Al acceder al directorio **/.git/** se comprobó que el repositorio era completamente accesible desde el exterior, permitiendo visualizar su estructura y potencialmente reconstruir el código fuente de la aplicación.

Evidencias

Comandos utilizados:

```
curl http://3.88.28.22:8000
```

```
curl http://3.88.28.22:8000/.git/
```

```
curl http://3.88.28.22:8000/.git/HEAD
```

```
curl http://3.88.28.22:8000/.git/config
```

Se obtuvo un listado del directorio `.git`, incluyendo elementos como:

HEAD

config

objects/

refs/

logs/

index

Asimismo, el archivo `HEAD` devolvía:

```
ref: refs/heads/master
```

confirmando que se trataba de un repositorio Git funcional.

Impacto

- Exposición del código fuente de la aplicación.
- Posible divulgación de credenciales, claves API o información sensible.
- Facilita el análisis de vulnerabilidades y el desarrollo de ataques dirigidos.

Severidad

Alta


```

paloma@kali: ~/git
Session Acciones Editar Vista Ayuda

2026-07-02 19:02:37 (23,6 KB/s) - «3.88.28.22:8000/.git/refs/heads/master» guardado [41/41]

--2026-07-02 19:02:37-- http://3.88.28.22:8000/.git/logs/refs/heads/master
Reutilizando la conexión con 3.88.28.22:8000.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 176 [application/octet-stream]
Grabando a: «3.88.28.22:8000/.git/logs/refs/heads/master»

3.88.28.22:8000/.git/logs/refs/heads/mast 100%[>] 176 --.-KB/s en 0s

2026-07-02 19:02:37 (2,93 MB/s) - «3.88.28.22:8000/.git/logs/refs/heads/master» guardado [176/176]

ACABADO --2026-07-02 19:02:37--
Tiempo total de reloj: 4,8s
Descargados: 45 ficheros, 35K en 0,01s (3,36 MB/s)

paloma@kali)~/git
$ grep -R "FLAG" .
paloma@kali)~/git
$ grep -R "password" .
$ grep -R "secret" .
$ grep -R "token" .
$ grep -R "api" .
./hooks/fsmonitor-watchman.sample: # The hook is passed a version (currently 2) and last update token
./hooks/fsmonitor-watchman.sample: # formatted as a string and outputs to stdout a new update token and
./hooks/fsmonitor-watchman.sample: # all files that have been modified since the update token. Paths must
./hooks/fsmonitor-watchman.sample: my ($version, $last_update_token) = @ARGV;
./hooks/fsmonitor-watchman.sample: # print STDERR "$0 $version $last_update_token\n";
./hooks/fsmonitor-watchman.sample: # changed since $last_update_token but not from the .git folder.
./hooks/fsmonitor-watchman.sample: if (substr($last_update_token, 0, 1) eq "c") {
./hooks/fsmonitor-watchman.sample: $last_update_token = "\$last_update_token";
./hooks/fsmonitor-watchman.sample: $last_update_line = qq[\n"since": $last_update_token,];
./hooks/fsmonitor-watchman.sample: $last_update_token = $0->{clock};
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: # The hook is passed a version (currently 2) and last update token
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: # formatted as a string and outputs to stdout a new update token and
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: # all files that have been modified since the update token. Paths must
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: my ($version, $last_update_token) = @ARGV;
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: # print STDERR "$0 $version $last_update_token\n";
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: # changed since $last_update_token but not from the .git folder.
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: if (substr($last_update_token, 0, 1) eq "c") {
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: $last_update_token = "\$last_update_token";
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: $last_update_line = qq[\n"since": $last_update_token,];
./3.88.28.22:8000/.git/hooks/fsmonitor-watchman.sample: $last_update_token = $0->{clock};
paloma@kali)~/git

```

```

paloma@kali: ~/git
Session Acciones Editar Vista Ayuda

21110/tcp closed unknown
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 232.23 seconds

paloma@kali)~/git
$ curl http://3.88.28.22:8000
<doctype html>
<html lang="en"><head><meta charset="utf-8"><title>Legacy Billing</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
<h1>Legacy Billing App</h1>
<p>Deprecated internal billing tool. Should have been decommissioned in 2019.</p>
<p>Nothing to see here. (Except the entire source repository - this directory
was deployed straight from a working git checkout.)</p>
</body></html>

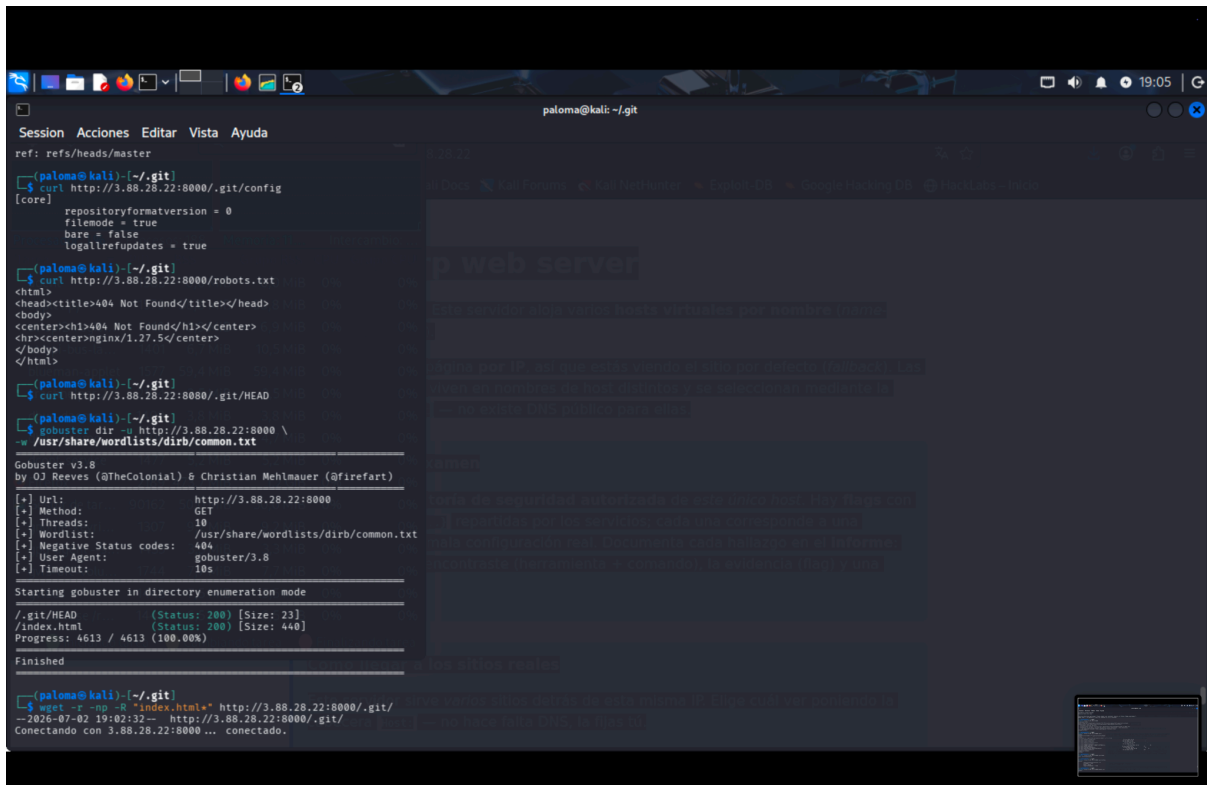
paloma@kali)~/git
$ curl http://3.88.28.22:8000/.git/
<html>
<head><title>Index of /.git/</title></head>
<body>
<h1>Index of /.git/</h1><pre><a href="..">../</a>
<a href="hooks">hooks/</a>
<a href="info">info/</a>
<a href="logs">logs/</a>
<a href="objects">objects/</a>
<a href="refs">refs/</a>
<a href="COMMIT_EDITMSG">COMMIT_EDITMSG</a>
<a href="HEAD">HEAD</a>
<a href="config">config</a>
<a href="description">description</a>
<a href="index">index</a>
</pre></body>
</html>

paloma@kali)~/git
$ curl http://3.88.28.22:8000/.git/HEAD
ref: refs/heads/master

paloma@kali)~/git
$ curl http://3.88.28.22:8000/.git/config
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true

paloma@kali)~/git
$ curl http://3.88.28.22:8000/robots.txt
<html>

```



```
Session Acciones Editar Vista Ayuda
ref: refs/heads/master

(paloma@kali) ~/git
$ curl http://3.88.28.22:8000/.git/config
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true

(paloma@kali) ~/git
$ curl http://3.88.28.22:8000/robots.txt
html
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<h2><center>nginx/1.27.5</center>
</body>
</html>

(paloma@kali) ~/git
$ curl http://3.88.28.22:8000/.git/HEAD
HEAD
ref: refs/heads/master

(paloma@kali) ~/git
$ gobuster dir -u http://3.88.28.22:8000 \
-w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://3.88.28.22:8000
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

./git/HEAD (Status: 200) [Size: 23]
./index.html (Status: 200) [Size: 440]
Progress: 4613 / 4613 (100.00%)
Finished

(paloma@kali) ~/git
$ wget -O - -r "index.html" http://3.88.28.22:8000/.git/
--2026-07-02 19:02:32-- http://3.88.28.22:8000/.git/
Conectando con 3.88.28.22:8000 ... conectado.
```

Recomendaciones

- Eliminar el directorio **.git** de los entornos de producción.
- Configurar el servidor web para impedir el acceso a archivos y directorios ocultos.
- Revisar que los despliegues de aplicaciones no incluyan repositorios de control de versiones.

F-11 – Exposición de un panel de monitorización sin autenticación

Descripción

Durante el escaneo completo de puertos mediante **Nmap** se identificó un servicio HTTP accesible en el puerto **8080**. Al acceder a dicho servicio se comprobó que correspondía a un panel de monitorización interno (**MegaCorp Monitoring**) expuesto públicamente y accesible sin ningún mecanismo de autenticación. Además, el propio servicio indicaba que el listado de directorios se encontraba habilitado, lo que incrementa la exposición de información sensible.

Evidencias

Herramienta utilizada:

```
nmap -Pn -p- -sV 3.88.28.22
```

Se detectó el siguiente servicio:

```
8080/tcp open http Caddy httpd
```

Posteriormente se accedió al servicio mediante:

```
curl http://3.88.28.22:8080
```

Obteniéndose la siguiente respuesta:

```
MegaCorp Monitoring
```

```
Internal monitoring dashboard, exposed on a non-standard port  
(8080) with no authentication.
```

```
FLAG{exposed_monitoring_8080}
```

```
Directory listing is enabled - see /status.txt
```

Impacto

- Acceso no autorizado a un panel de monitorización interno.
- Exposición de información operativa de la infraestructura.
- Incremento de la superficie de ataque al publicar servicios administrativos accesibles desde Internet.
- La existencia de listado de directorios puede facilitar la obtención de información adicional por parte de un atacante.

Severidad

Alta

```
(paloma@kali)~/.git
$ grep -R "FLAG{" .

(paloma@kali)~/.git
$ cat .git/logs/HEAD
cat: .git/logs/HEAD: No existe el fichero o el directorio

(paloma@kali)~/.git
$ cat .git/COMMIT_EDITMSG
cat: .git/COMMIT_EDITMSG: No existe el fichero o el directorio

(paloma@kali)~/.git
$ curl http://3.88.28.22:8080
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>MegaCorp Monitoring</title></head>
<body style="font-family:sans-serif;max-width:720px;margin:3rem auto">
  <h1>MegaCorp Monitoring</h1>
  <p>Internal monitoring dashboard, exposed on a non-standard port (8080) with
  no authentication.</p>
  <p><strong>FLAG{exposed_monitoring_8080}</strong></p>
  <ul>
    <li>Directory listing is enabled – see <a href="/status.txt">status.txt</a></li>
  </ul>
</body></html>

(paloma@kali)~/.git
$
```

Figura 18. Descubrimiento de un panel de monitorización expuesto en el puerto 8080

La imagen muestra el acceso al servicio HTTP identificado durante el escaneo completo de puertos mediante Nmap. Al conectarse al puerto 8080 utilizando la herramienta **curl**, se comprobó que el servidor alojaba un panel interno de monitorización denominado MegaCorp Monitoring, accesible públicamente sin requerir autenticación. La respuesta del servidor incluía la FLAG{exposed_monitoring_8080}, confirmando la vulnerabilidad, y además indicaba que el listado de directorios estaba habilitado, remitiendo al recurso /status.txt, lo que evidencia una configuración insegura que puede facilitar la divulgación de información sensible sobre la infraestructura y aumentar la superficie de ataque del sistema.

Recomendaciones

- Restringir el acceso al panel de monitorización mediante autenticación robusta.
- Limitar el acceso al servicio mediante firewall, VPN o listas de control de acceso (ACL).
- Deshabilitar el listado de directorios en el servidor web.
- Evitar exponer servicios internos en puertos accesibles públicamente y realizar auditorías periódicas para detectar este tipo de configuraciones inseguras.

6. CADENA DE ATAQUE (NARRATIVA)

La auditoría comenzó con la fase de reconocimiento, durante la cual se identificó la existencia de varios **hosts virtuales** y se obtuvo información relevante del **certificado TLS (F-01, F-02 y F-08)**. La información contenida en los *Subject Alternative Names (SAN)* permitió descubrir nuevos dominios internos no visibles inicialmente.

Posteriormente, mediante la modificación de la cabecera **Host**, fue posible acceder a **hosts virtuales internos (F-09)**, obteniendo información sensible sobre la infraestructura y descubriendo la existencia de una aplicación de facturación heredada y un repositorio Git desplegado accidentalmente.

A continuación, el escaneo completo de puertos permitió identificar servicios adicionales, entre ellos una **aplicación Legacy Billing** con el directorio **.git** expuesto (**F-10**) y un **panel de monitorización interno** accesible sin autenticación en el puerto **8080 (F-11)**. La combinación de estos hallazgos permitió ampliar progresivamente el conocimiento de la infraestructura, localizar recursos internos y demostrar que múltiples servicios críticos eran accesibles desde Internet debido a configuraciones inseguras y a la ausencia de medidas adecuadas de hardening.

Cadena de ataque resumida:

F-01 → F-02 → F-08 → F-09 → F-10 → F-11 → Obtención de información sensible, acceso a recursos internos y ampliación de la superficie de ataque mediante múltiples errores de configuración.

7. CONCLUSIONES Y RECOMENDACIONES

Postura de seguridad general

La infraestructura auditada presenta una **postura de seguridad deficiente** debido a la acumulación de errores de configuración y exposición de información sensible. Se identificaron múltiples vulnerabilidades que permitieron descubrir hosts virtuales internos, acceder a servicios restringidos, exponer repositorios Git, obtener información de la infraestructura mediante certificados TLS y acceder a servicios administrativos sin autenticación. La combinación de estos hallazgos facilita considerablemente las tareas de reconocimiento y aumenta la superficie de ataque del sistema.

Patrones de raíz comunes

- Exposición de información sensible durante la fase de reconocimiento.
- Configuración insegura de servidores web.
- Exposición de repositorios Git y archivos internos.
- Falta de hardening en los servicios publicados.
- Ausencia de mecanismos de autenticación en servicios internos.
- Exposición de directorios y recursos mediante listado de directorios.
- Divulgación de información de la infraestructura a través de certificados TLS.

Plan de remediación priorizado

Prioridad	Acción	Hallazgos que mitiga	Esfuerzo
1	Eliminar todos los repositorios .git y cualquier archivo sensible de los entornos de producción.	F-03, F-10	Bajo
2	Restringir mediante autenticación, firewall o VPN el acceso a hosts virtuales y servicios internos.	F-09, F-11	Medio
3	Deshabilitar el listado de directorios y proteger los recursos publicados en los servidores web.	F-06, F-10, F-11	Bajo

4	Revisar la configuración TLS para eliminar nombres internos de los certificados y utilizar certificados emitidos por una CA de confianza.	F-08	Bajo
5	Aplicar medidas de hardening en todos los servicios expuestos, eliminando aplicaciones obsoletas y reduciendo la superficie de ataque.	F-01, F-02, F-07, F-10, F-11	Medio
6	Implantar un procedimiento de revisión de seguridad previo al despliegue y realizar auditorías periódicas de configuración y exposición de servicios.	Todos los hallazgos (F-01 a F-11)	Medio

8. ANEXOS

Durante la fase de reconocimiento y enumeración se emplearon herramientas estándar de auditoría de seguridad junto con las wordlists incluidas por defecto en Kali Linux.

Herramientas utilizadas

- Nmap (escaneo de puertos y detección de servicios).
- cURL (peticiones HTTP y modificación de la cabecera **Host**).
- OpenSSL (inspección del certificado TLS).
- Gobuster (enumeración de directorios y recursos web).
- Git (análisis del repositorio expuesto).
- Wget (descarga de recursos del servidor).

Wordlists utilizadas

- `/usr/share/wordlists/dirb/common.txt`

Esta wordlist se utilizó con **Gobuster** para identificar directorios y recursos accesibles en los servicios HTTP expuestos.

Scripts propios

Se utilizó un pequeño script en Bash para automatizar la enumeración de hosts virtuales obtenidos del certificado TLS:

```
for h in www.megacorp.lab dev.megacorp.lab admin.megacorp.lab api.megacorp.lab
internal.megacorp.lab staging.megacorp.lab vpn.megacorp.lab; do
    echo "==== $h ====="
    curl -ks -H "Host: $h" https://3.88.28.22 | head -30
    echo
done
```

Este script permitió identificar los distintos **Virtual Hosts** publicados en el servidor, localizar recursos internos y descubrir varias de las *FLAGS* del ejercicio.